

Prueba a lo Largo del Ciclo de Vida de Desarrollo de Software

Cap. 2 | Fundamentos de pruebas de software

2

El **modelo de ciclo de vida de desarrollo de software (CVDS)** es una representación abstracta y de alto nivel del proceso de desarrollo de software. Un modelo CVDS define cómo se relacionan entre sí, tanto lógica como cronológicamente, las diferentes fases de desarrollo y los tipos de actividades que se realizan.

MODELOS DE CICLO DE DESARROLLO DE SOFTWARE

Los modelos de ciclo de vida de desarrollo de software más comunes se clasifican en:

- **Modelos de desarrollo secuenciales:** cada fase comienza cuando la anterior ha finalizado, como el modelo en cascada o el modelo V.
- **Modelos de desarrollo iterativos e incrementales:** se repiten las fases tantas veces como sea necesario, como el modelo en espiral, el prototipado.

Algunas actividades del proceso pueden describirse mediante los métodos de desarrollo más detallados y prácticas ágiles, por ejemplo

- **DGPA o ATDD:** Desarrollo guiado por pruebas de aceptación
- **DGC o BDD:** Desarrollo guiado por comportamiento
- **DGD o DDD:** Diseño guiado por dominios
- **XP:** Programación Extrema
- **DGP:** Desarrollo guiado por prestaciones
- Kanban
- Lean IT
- Scrum y
- **DGP o TDD:** Desarrollo guiado por pruebas

BUENAS PRÁCTICAS DE PRUEBAS

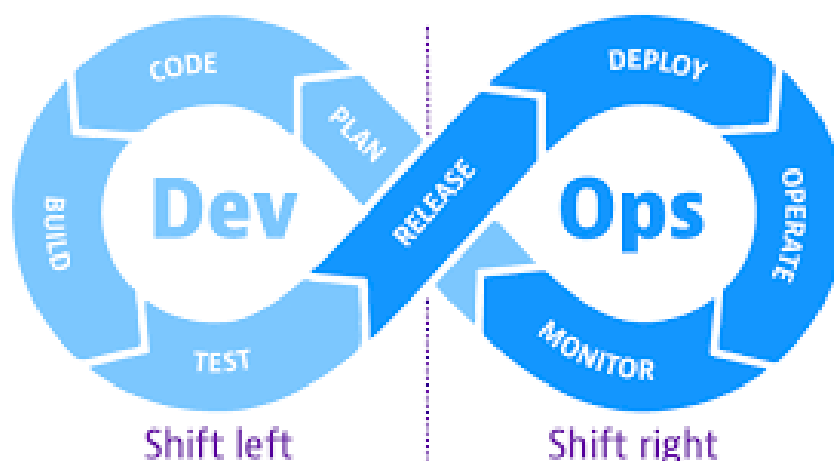
Independientemente del modelo de CVDS elegido,

- Cada actividad de desarrollo tiene su correspondiente actividad de prueba
- Cada nivel de prueba tiene objetivos específicos y diferentes.
- El análisis y diseño de pruebas para un nivel comienza durante la fase de desarrollo correspondiente al CVDS (prueba temprana)
- Los probadores participan de la revisión de productos de trabajo tan pronto como están listos los borradores (shift-left)



Las pruebas deben adaptarse al CVDS:

- Alcance y cronología
- Nivel de detalle de la documentación
- Elección de técnicas y enfoque de pruebas.
- Alcance de automatizaciones
- Rol y responsabilidades.





La Prueba como **Impulsor del Desarrollo de Software**

Desarrollo Guiado por Prueba, desarrollo guiado por prueba de aceptación y desarrollo guiado por comportamiento son enfoques de desarrollo similares, donde la prueba es el medio para conducir el desarrollo.

Desarrollo guiado prueba

- Dirige la codificación mediante casos de prueba
- Primero se redactan las pruebas y luego se elabora el código que las satisfaga.

Desarrollo guiado por prueba de aceptación

- Obtiene pruebas a partir de criterio de aceptación como parte del proceso de diseño del sistema
- Las pruebas se redactan antes de que la parte de la aplicación se desarrolle para satisfacer las pruebas.

Desarrollo guiado por comportamiento

- Expresa el comportamiento deseado de una aplicación con casos de prueba escritos en forma sencilla (Gherkin)
- A continuación, los casos de prueba se traducen automáticamente en pruebas ejecutables.



DevOps y La prueba

DevOps es un enfoque organizativo que pretende crear sinergias haciendo que el desarrollo y las operaciones trabajen juntos para alcanzar objetivos comunes. Promueve la autonomía de equipos, retroalimentación rápida, cadenas de herramientas integradas y prácticas como integración y entrega continuas. Desde el punto de vista de las pruebas

- Rápida retroalimentación sobre la calidad del código
- IC promueve el shift-left con código de alta calidad con pruebas de componentes y análisis estático.
- Promueve procesos automatizados
- Reduce la necesidad de pruebas manuales repetitivas
- Minimiza el riesgo de regresión

Riesgos y desafíos

- El pipeline de entrega DevOps debe ser definido y establecido
- Las herramientas de CI/CD deben ser introducidas y mantenidas
- La automatización de la prueba requiere recursos adicionales y puede ser difícil de establecer y mantener.

Las pruebas manuales en DevOps siguen siendo necesarias

ENFOQUE “DESPLAZAMIENTO A LA IZQUIERDA”

La prueba se realiza en forma temprana en el CVDS. Sugiere que la prueba debería realizarse antes.

BUENAS PRÁCTICAS

- Revisión de la especificación desde la perspectiva de la prueba.
- Redactar casos de pruebas antes de escribir el código y hacer que este se ejecute en un arnés de prueba durante la implementación de código
- Utilizar integración continua e incluso mejor, entrega continua, ya que aporta retroalimentación rápida y prueba de componentes automatizadas.
- Completar el análisis estático del código fuente antes de la prueba dinámica
- Realizar pruebas no funcionales empezando por el nivel de componentes, siempre que sea posible.



Atención

- Un enfoque de desplazamiento a la izquierda puede dar lugar a formación, esfuerzo y/o costes adicionales al principio del proceso, pero se espera que ahorre esfuerzos y/o costes con posterioridad
- Para aplicar este enfoque es importante que los implicados estén convencidos del camino a tomar y asuman los beneficios y esfuerzos que asume

RETROSPECTIVAS Y MEJORA DEL PROCESO

Suelen celebrarse al final de un proyecto o de una iteración. En un hito de entrega o pueden celebrarse cuando se necesiten. Cuándo y cómo se realizan dependen del modelo de CVDS

BENEFICIOS

- Aumento de la efectividad/eficiencia de la prueba
- Aumento de la calidad de los productos de prueba
- Unión y aprendizaje en equipo
- Mejora de la calidad de la base de prueba
- Mejor cooperación entre el desarrollo y la prueba.



Importante

- De las reuniones participan no sólo probadores
- Los resultados deben registrarse.
- Fundamentales para la implementación de mejora continua.
- Importante para el seguimiento de cualquier mejora recomendada.

NIVELES DE PRUEBA

Los **Niveles** de prueba son grupos de actividades de prueba que se organizan y gestionan conjuntamente.

Están relacionados con otras actividades dentro de CVDS. En los modelos secuenciales, los niveles de prueba suelen definirse de tal forma que los criterios de salida de un nivel forman los criterios de entrada del siguiente.

Nivel	Objetivos	Objetos de prueba	Bases de prueba	Defectos típicos	Responsables
Prueba de componentes	Conseguir defectos en objetos de prueba. Reducir el riesgo. Construir confianza en la calidad del componente. Prevenir defectos que escalen a niveles de prueba más altos.	Código / Estructuras de datos. Clases. Módulos de bases de datos.	Especificaciones de componentes. Código. Modelo de datos.	Funcionamiento no acorde a la especificación. Problemas de flujos de datos. Código y lógica incorrecta.	Realizado por el desarrollador.
Prueba de Integración de Componentes	Conseguir defectos y fallas entre las interfaces de módulos de sistemas, así como en los módulos en sí mismos. Reducir el riesgo. Construir confianza en la calidad de las interfaces. Prevenir defectos que escalen a niveles de prueba más altos.	Subsistemas. Bases de datos. Infraestructura. Interfaces. Interfaces de comunicación de aplicaciones API's. Microservicios	Diseño de software y sistemas. Diagrama de secuencias. Especificaciones de interfaz y protocolos de comunicación. Casos de uso. Arquitectura a nivel de componentes o sistemas. Flujos de trabajo. Definiciones de interfaces externas.	Datos incorrectos, faltantes o mal codificados. Secuenciación o sincronización incorrecta a llamadas de interfaz. Incompatibilidad de la interfaz. Fallos en la comunicación de componentes manejados inadecuadamente o no manejados en absoluto. Suposiciones incorrectas del significado, unidades o límites de datos transmitidos entre componentes.	Las pruebas de integración de componentes son a menudo responsabilidad de los desarrolladores. Las pruebas de integración de sistemas son responsabilidad de los probadores.
Prueba de Sistema	Reducir el riesgo. Verificar que los requerimientos del sistema son los diseñados y a su vez validados con lo especificado por el usuario. Validar que el sistema está completo y funciona como se esperaba. Generar confianza en la calidad del sistema en su conjunto. Encontrar defectos y fallas. Evitar que los defectos escapen a niveles de pruebas más altos, como las pruebas de aceptación o producción.	Aplicaciones. Sistemas de hardware y software. Sistemas operativos. Sistemas bajo prueba. Configuración del sistema y datos de configuración.	Especificaciones de requisitos del sistema. Informes de análisis de riesgo. Casos de uso. Épicas e historias de usuarios. Modelos de comportamiento del sistema. Diagramas de estado. Manuales de sistema y de usuario.	Cálculos incorrectos. Comportamiento incorrecto de tareas a través del sistema. Control de datos o flujos de datos incorrectos dentro del sistema. Funcionamiento incorrecto del sistema.	Realizadas generalmente por probadores independientes.

Nivel	Objetivos	Objetos de prueba	Bases de prueba	Defectos típicos	Responsables
Prueba de Aceptación	Establecer confianza en la calidad del sistema como un todo. Validar que el sistema está completo y funciona como se espera. Verificar que los requerimientos funcionales y no funcionales del sistema cumplen con las especificaciones. Satisfacer requisitos legales o regulatorios.	Sistema bajo prueba. Configuración del sistema y datos de configuración. Procesos empresariales para un sistema totalmente integrado. Sistemas de recuperación y sitios activos Procesos operativos y de mantenimiento. Formularios. Reportes Datos de producción.	Generales: Procesos de negocio Requisitos del usuario o del negocio. Regulaciones, contratos legales y estándares. Casos de uso. Requisitos del sistema. Documentación del sistema o del usuario. Procedimientos de Instalación Informes de análisis de riesgos Operativas: Procedimientos de respaldo y restauración. Procedimientos de recuperación ante Desastres. Requisitos no funcionales. Documentación de Operaciones. Instrucciones de despliegue e instalación. Objetivos de rendimiento. Paquetes de bases de datos. Normas o regulaciones de seguridad	Las reglas de negocio no se implementan Correctamente. El sistema no cumple los requisitos contractuales o reglamentarios. Fallas no funcionales	Clientes, usuarios de negocio, propietarios de productos o los operadores de un sistema y otras partes interesadas

Niveles de prueba:

Tipos de pruebas de aceptación

Prueba de Aceptación de Usuario:

Se centra en validar que el sistema cumple los requisitos de funcionamiento esperado en un entorno operativo real o simulado.

Prueba de Aceptación Operativa:

Valida si el sistema cumple con los requisitos de operación y la realizan los usuarios y los administradores de la aplicación.

Prueba de Aceptación Contractual:

Se realiza según los criterios de aceptación de un contrato para producir software desarrollado a medida.

Prueba Alfa y Beta:

Se realizan para recibir retroalimentación de usuarios, clientes y/u operadores potenciales o existentes antes de la liberación del producto de software al mercado.

Prueba Alfa:

Se realizan en las instalaciones de la organización que desarrolla, pero no por el equipo de desarrollo.

Pruebas Beta:

Se realiza en las propias ubicaciones del cliente, fuera de la organización que desarrolla.

TIPOS DE PRUEBA

Pruebas Funcionales: se encargan del "qué" debe hacer el sistema.

Pruebas No Funcionales: prueban las características del producto de software, qué tan bien se ejecutan elementos como: recuperación, mantenibilidad, seguridad, resistencia a fallos, volumen, estrés y carga.

Pruebas de Caja Blanca: se enfocan en la estructura interna del sistema, entiéndase el código, arquitectura o flujos de datos dentro del sistema. Suele ejecutarse en el nivel de Componente e Integración.

Pruebas Relacionadas a Cambios:

Pruebas de Confirmación: garantiza que el defecto reparado pase las pruebas que previamente falló.

Pruebas de Regresión: garantizan que no se hayan generado nuevos problemas secundarios como consecuencia de los ajustes realizados al software.

Pruebas de Mantenimiento:

Sucedan cuando se realizan cambios como parte del mantenimiento del software, evalúan el éxito del cambio en sí mismo y verifican posibles efectos secundarios. Es decir, constan de dos partes:

Probar los cambios y, Pruebas de regresión.

Disparadores de Mantenimiento son modificaciones, migración y retiro del **software**.

Análisis de impacto evalúa las consecuencias de la implementación de un cambio en el **software**.

